

11. Vulnerability Assessment: Identify and analyze system vulnerabilities using basic security tools

Title

Vulnerability Assessment: Identify and Analyze System Vulnerabilities Using Basic Security Tools

Aim

To understand the fundamentals of vulnerability assessment and identify potential security weaknesses in a controlled environment using Python-based security assessment techniques in Google Colab.

Objectives

After completing this practical, students will be able to:

1. Understand the concept of vulnerability assessment.
 2. Differentiate between vulnerability, threat, risk, and attack.
 3. Understand basic vulnerability assessment methodology.
 4. Identify security weaknesses from simulated system information.
 5. Analyze open ports, outdated services, weak configurations, and suspicious settings.
 6. Apply basic rule-based vulnerability detection techniques.
 7. Classify vulnerabilities according to severity.
 8. Calculate a basic risk score.
 9. Generate a vulnerability assessment report.
 10. Visualize identified vulnerabilities using Python.
 11. Understand the importance of vulnerability remediation.
 12. Perform a basic vulnerability assessment using Google Colab.
-

1. Introduction

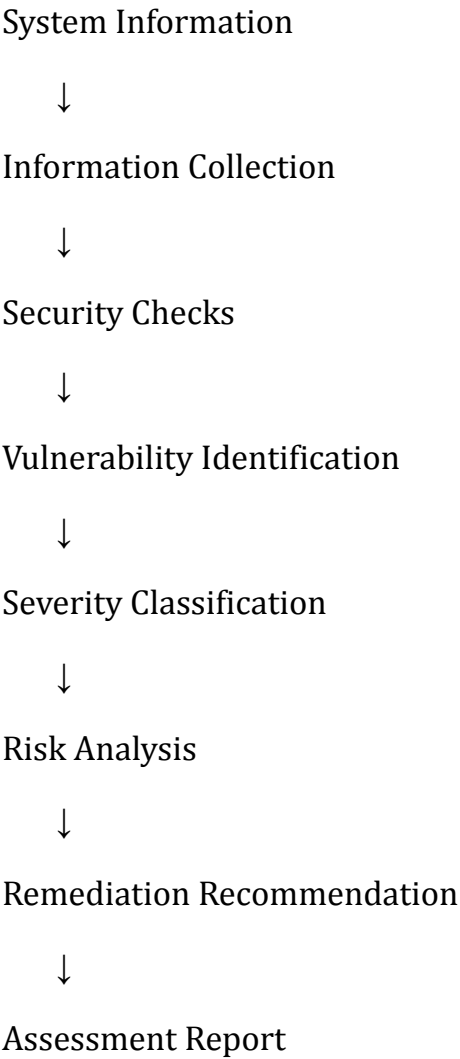
A **vulnerability** is a weakness or flaw in a system, application, network, configuration, or security control that could potentially be exploited to compromise confidentiality, integrity, or availability.

Examples include:

- Unnecessary open ports
- Outdated software
- Weak passwords
- Insecure services
- Misconfigured access controls
- Missing security updates
- Unencrypted communication
- Excessive user privileges

A **Vulnerability Assessment** is the systematic process of identifying, analyzing, prioritizing, and reporting security weaknesses.

Basic Vulnerability Assessment Process



2. Vulnerability vs Threat vs Risk

Term	Meaning
Vulnerability	A weakness in a system
Threat	A potential source of harm
Attack	An attempt to exploit a weakness

Risk	Potential impact resulting from a threat exploiting a vulnerability
-------------	---

Example

Weak Password



Vulnerability

Attacker



Threat

Password Guessing Attempt



Attack

Possible Account Compromise



Risk

3. Basic Vulnerability Assessment Techniques

For this practical, students will simulate several basic assessment techniques.

Technique 1: Port Assessment

Check whether unnecessary or risky ports are open.

Example:

Port 22 → SSH

Port 80 → HTTP

Port 443 → HTTPS

Port 23 → Telnet

Telnet can be flagged because it is an insecure legacy protocol when used without additional protection.

Technique 2: Service Version Assessment

Identify whether a service is using an outdated version.

Example:

Apache 2.2

OpenSSH 7.2

The program will flag versions that are defined as outdated in the simulation rules.

Technique 3: Password Policy Assessment

Check whether the system follows basic password-security requirements.

Example:

Minimum password length < 8

Password complexity disabled

Technique 4: Encryption Assessment

Check whether sensitive services use encryption.

Example:

HTTP → Unencrypted

HTTPS → Encrypted

Technique 5: Privilege Assessment

Check for excessive administrative privileges.

Example:

Administrator accounts = 8

Expected maximum = 3

This may indicate a security configuration concern.

4. Google Colab Implementation

The following is a **complete single-cell program**. Students can copy the entire code into **one Google Colab cell** and execute it.

Safety: This practical uses **synthetic system information**. It does not scan Google Colab's infrastructure, external IP addresses, websites, or unauthorized systems.

```
# =====
```

```
# PRACTICAL 11
```

```
# VULNERABILITY ASSESSMENT
```

```
#
```

```
# Aim:
```

```
# Identify and analyze system vulnerabilities using basic
```

```
# security assessment techniques in a controlled environment.
```

```
#
```

```
# Platform:
```

```
# Google Colab
```

```
#
```

```
# IMPORTANT:
```

```
# This program uses SYNTHETIC SYSTEM DATA.
```

```
# It does NOT scan real systems or external networks.
```

```
# =====
```

```
# =====
```

```
# 1. IMPORT REQUIRED LIBRARIES
```

```
# =====
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
print("=" * 75)
```

```
print("VULNERABILITY ASSESSMENT - GOOGLE COLAB SIMULATION")
```

```
print("=" * 75)
```

```
print("\nRequired libraries imported successfully.")
```

```
# =====
```

```
# 2. SIMULATED SYSTEM INFORMATION
```

```
# =====
```

```
system_info = {
```

```
    "system_name": "Demo-Web-Server",
```

```
    "operating_system": "Linux",
```

```
    "os_version": "20.04",
```

```
    "hostname": "demo-server",
```

```
    "admin_accounts": 7,
```

```
    "minimum_password_length": 6,
```

```
    "password_complexity": False,
```

```
    "firewall_enabled": False,
```

```
    "automatic_updates": False,
```

```
    "encryption_enabled": False
}
```

```
print("\nSYSTEM INFORMATION")
print("-" * 75)
```

```
for key, value in system_info.items():
```

```
    print(f"{key:<30} : {value}")
```

```
# =====
# 3. SIMULATED OPEN PORTS AND SERVICES
# =====
```

```
services = [
```

```
    {
        "port": 22,
        "protocol": "TCP",
        "service": "SSH",
        "version": "OpenSSH 7.2",
        "status": "Open"
    },
```

```
    {
```

```
"port": 23,  
"protocol": "TCP",  
"service": "Telnet",  
"version": "Telnet 1.0",  
"status": "Open"  
},
```

```
{  
  "port": 80,  
  "protocol": "TCP",  
  "service": "HTTP",  
  "version": "Apache 2.2",  
  "status": "Open"  
},
```

```
{  
  "port": 443,  
  "protocol": "TCP",  
  "service": "HTTPS",  
  "version": "Apache 2.4",  
  "status": "Open"  
},
```

```
{  
  "port": 3306,  
  "protocol": "TCP",  
  "service": "MySQL",  
  "version": "MySQL 5.5",
```



```
        "status": "Open"
    },

    {
        "port": 8080,
        "protocol": "TCP",
        "service": "Web Application",
        "version": "Demo 1.0",
        "status": "Open"
    }
]
```

```
services_df = pd.DataFrame(services)
```

```
print("\n" + "=" * 75)
```

```
print("SIMULATED SERVICES AND PORTS")
```

```
print("=" * 75)
```

```
display(services_df)
```

```
# =====
```

```
# 4. VULNERABILITY RESULT STORAGE
```

```
# =====
```

```
vulnerabilities = []
```

```
# =====
```

```
# 5. PORT-BASED VULNERABILITY CHECK
```

```
# =====
```

```
for _, row in services_df.iterrows():
```

```
    port = row["port"]
```

```
    service = row["service"]
```

```
    # Telnet
```

```
    if port == 23:
```

```
        vulnerabilities.append({
```

```
            "Category": "Network Service",
```

```
            "Finding": "Telnet service is enabled",
```

```
            "Evidence": f"Port {port} ({service}) is open",
```

```
            "Severity": "HIGH",
```

```
            "Risk Score": 8,
```

```
            "Recommendation":
```

```
                "Disable Telnet and use a secure remote administration protocol."
```

```
}}
```

```
# MySQL exposed
```

```
elif port == 3306:
```

```
    vulnerabilities.append({
```

```
        "Category": "Database Exposure",
```

```
        "Finding": "Database service is externally exposed",
```

```
        "Evidence": f"Port {port} ({service}) is open",
```

```
        "Severity": "HIGH",
```

```
        "Risk Score": 8,
```

```
        "Recommendation":
```

```
            "Restrict database access to trusted hosts or internal networks."
```

```
    })
```

```
# =====
```

```
# 6. OUTDATED SERVICE CHECK
```

```
# =====
```

```
for _, row in services_df.iterrows():
```

```
    version = row["version"]
```

```
    if "Apache 2.2" in version:
```

```
        vulnerabilities.append({
```

```
            "Category": "Software Version",
```

```
            "Finding": "Outdated web server version",
```

```
            "Evidence": version,
```

```
            "Severity": "HIGH",
```

```
            "Risk Score": 8,
```

```
            "Recommendation":
```

```
                "Upgrade the web server to a supported and patched version."
```

```
        })
```

```
    elif "OpenSSH 7.2" in version:
```

```
        vulnerabilities.append({
```

"Category": "Software Version",

"Finding": "Outdated SSH version",

"Evidence": version,

"Severity": "MEDIUM",

"Risk Score": 6,

"Recommendation":

"Upgrade SSH software and apply current security updates."

}}

elif "MySQL 5.5" in version:

vulnerabilities.append({

"Category": "Software Version",

"Finding": "Outdated database version",

"Evidence": version,

"Severity": "HIGH",

"Risk Score": 8,

"Recommendation":

"Upgrade the database to a supported version."

}}

=====

7. PASSWORD POLICY CHECK

=====

if system_info["minimum_password_length"] < 8:

vulnerabilities.append({

"Category": "Authentication",

"Finding": "Weak minimum password length",

"Evidence":

f"Minimum password length = "

f"{system_info['minimum_password_length']}",

"Severity": "MEDIUM",

"Risk Score": 6,

```
    "Recommendation":  
        "Use a minimum password length of at least 8 characters."  
    })
```

```
if system_info["password_complexity"] == False:
```

```
    vulnerabilities.append({
```

```
        "Category": "Authentication",
```

```
        "Finding": "Password complexity is disabled",
```

```
        "Evidence": "Password complexity = Disabled",
```

```
        "Severity": "MEDIUM",
```

```
        "Risk Score": 6,
```

```
        "Recommendation":
```

```
            "Enable password complexity requirements."  
    })
```

```
# =====
```

```
# 8. FIREWALL CHECK
```

```
# =====
```

```
if system_info["firewall_enabled"] == False:
```

```
    vulnerabilities.append({
```

```
        "Category": "Network Security",
```

```
        "Finding": "Firewall is disabled",
```

```
        "Evidence": "Firewall status = Disabled",
```

```
        "Severity": "HIGH",
```

```
        "Risk Score": 8,
```

```
        "Recommendation":
```

```
            "Enable and properly configure the system firewall."
```

```
    })
```

```
# =====
```

```
# 9. AUTOMATIC UPDATE CHECK
```

```
# =====
```

```
if system_info["automatic_updates"] == False:
```

```
    vulnerabilities.append({
```


"Category": "Patch Management",

"Finding": "Automatic security updates are disabled",

"Evidence": "Automatic updates = Disabled",

"Severity": "MEDIUM",

"Risk Score": 6,

"Recommendation":

"Enable automatic security updates or establish a patch-management process."

}}

=====

10. ENCRYPTION CHECK

=====

if system_info["encryption_enabled"] == False:

vulnerabilities.append({

"Category": "Data Protection",

"Finding": "Encryption protection is disabled",

"Evidence": "Encryption = Disabled",

```
"Severity": "HIGH",

"Risk Score": 8,

"Recommendation":
    "Enable encryption for sensitive data and communications."
})
```

```
# =====
# 11. ADMINISTRATIVE PRIVILEGE CHECK
# =====
```

```
if system_info["admin_accounts"] > 3:
```

```
    vulnerabilities.append({
```

```
        "Category": "Access Control",
```

```
        "Finding": "Excessive administrative accounts",
```

```
        "Evidence":
```

```
            f"Administrative accounts = "
```

```
            f"{system_info['admin_accounts']}",
```

```
        "Severity": "MEDIUM",
```

"Risk Score": 6,

"Recommendation":

"Review administrative accounts and apply least-privilege principles."

}}

```
# =====
```

```
# 12. CREATE VULNERABILITY DATAFRAME
```

```
# =====
```

```
vulnerability_df = pd.DataFrame(vulnerabilities)
```

```
print("\n" + "=" * 75)
```

```
print("IDENTIFIED VULNERABILITIES")
```

```
print("=" * 75)
```

```
display(vulnerability_df)
```

```
# =====
```

```
# 13. VULNERABILITY COUNT
```

```
# =====
```

```
total_vulnerabilities = len(vulnerability_df)
```

```
high_count = len(
```

```
vulnerability_df[
    vulnerability_df["Severity"] == "HIGH"
]
```

```
medium_count = len(
    vulnerability_df[
        vulnerability_df["Severity"] == "MEDIUM"
    ]
)
```

```
low_count = len(
    vulnerability_df[
        vulnerability_df["Severity"] == "LOW"
    ]
)
```

```
print("\n" + "=" * 75)
print("VULNERABILITY SUMMARY")
print("=" * 75)
```

```
print("\nTotal Vulnerabilities :", total_vulnerabilities)
```

```
print("High Severity      :", high_count)
```

```
print("Medium Severity     :", medium_count)
```

```
print("Low Severity      :", low_count)
```

```
# =====
```

```
# 14. RISK SCORE ANALYSIS
```

```
# =====
```

```
if total_vulnerabilities > 0:
```

```
    total_risk = vulnerability_df["Risk Score"].sum()
```

```
    average_risk = vulnerability_df["Risk Score"].mean()
```

```
else:
```

```
    total_risk = 0
```

```
    average_risk = 0
```

```
print("\n" + "=" * 75)
```

```
print("RISK ANALYSIS")
```

```
print("=" * 75)
```

```
print("\nTotal Risk Score :", total_risk)
```

```
print("Average Risk Score :", round(average_risk, 2))
```

```
# =====
```

```
# 15. OVERALL RISK CLASSIFICATION
```

```
# =====
```

```
if average_risk >= 7:
```

```
    overall_risk = "HIGH"
```

```
elif average_risk >= 4:
```

```
    overall_risk = "MEDIUM"
```

```
else:
```

```
    overall_risk = "LOW"
```

```
print("\nOverall Risk Level :", overall_risk)
```

```
# =====
```

```
# 16. CATEGORY-WISE ANALYSIS
```

```
# =====
```

```
category_summary = (
```

```
    vulnerability_df
```

```
    .groupby("Category")
```

```
.agg(  
    Vulnerabilities=("Finding", "count"),  
    Total_Risk=("Risk Score", "sum")  
)  
.reset_index()  
)
```

```
print("\n" + "=" * 75)  
print("CATEGORY-WISE VULNERABILITY ANALYSIS")  
print("=" * 75)
```

```
display(category_summary)
```

```
# =====  
# 17. SEVERITY DISTRIBUTION  
# =====
```

```
severity_counts = (  
    vulnerability_df["Severity"]  
    .value_counts()  
)
```

```
plt.figure(figsize=(8, 5))
```

```
severity_counts.plot(kind="bar")
```

```
plt.title("Vulnerability Severity Distribution")
```

```
plt.xlabel("Severity")
```

```
plt.ylabel("Number of Vulnerabilities")
```

```
plt.xticks(rotation=0)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====
```

```
# 18. CATEGORY DISTRIBUTION
```

```
# =====
```

```
category_counts = (  
    vulnerability_df["Category"]  
    .value_counts()  
)
```

```
plt.figure(figsize=(10, 5))
```

```
category_counts.plot(kind="bar")
```



```
plt.title("Vulnerabilities by Category")
```

```
plt.xlabel("Vulnerability Category")
```

```
plt.ylabel("Number of Vulnerabilities")
```

```
plt.xticks(rotation=30, ha="right")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====  
# 19. RISK SCORE VISUALIZATION  
# =====
```

```
plt.figure(figsize=(10, 5))
```

```
plt.bar(  
    range(len(vulnerability_df)),  
    vulnerability_df["Risk Score"]  
)
```

```
plt.title("Risk Score of Identified Vulnerabilities")
```

```
plt.xlabel("Vulnerability")
```

```
plt.ylabel("Risk Score")
```

```
plt.xticks(  
    range(len(vulnerability_df)),  
    range(1, len(vulnerability_df) + 1)  
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====
```

```
# 20. HIGH-RISK FINDINGS
```

```
# =====
```

```
high_risk_findings = vulnerability_df[  
    vulnerability_df["Severity"] == "HIGH"  
]
```

```
print("\n" + "=" * 75)
```

```
print("HIGH-RISK VULNERABILITIES")
```

```
print("=" * 75)
```

```
if len(high_risk_findings) > 0:
```

```
display(  
    high_risk_findings[  
        [  
            "Category",  
            "Finding",  
            "Evidence",  
            "Risk Score",  
            "Recommendation"  
        ]  
    ]  
)
```

else:

```
print("\nNo high-risk vulnerabilities identified.")
```

```
# =====
```

```
# 21. REMEDIATION REPORT
```

```
# =====
```

```
print("\n" + "=" * 75)
```

```
print("VULNERABILITY REMEDIATION REPORT")
```

```
print("=" * 75)
```

```
for i, row in vulnerability_df.iterrows():
```

```
print(f"\n{i + 1}. {row['Finding']}")
```

```
print(f"    Category      : {row['Category']}")
```

```
print(f"    Evidence       : {row['Evidence']}")
```

```
print(f"    Severity        : {row['Severity']}")
```

```
print(f"    Risk Score       : {row['Risk Score']}")
```

```
print(f"    Recommendation : {row['Recommendation']}")
```

```
# =====  
# 22. EXPORT ASSESSMENT REPORT  
# =====
```

```
report_file = "vulnerability_assessment_report.csv"
```

```
vulnerability_df.to_csv(  
    report_file,  
    index=False  
)
```

```
print("\n" + "=" * 75)
```

```
print(
```

```
"Assessment report saved as:",  
report_file  
)
```

```
# =====
```

```
# 23. FINAL SECURITY ASSESSMENT
```

```
# =====
```

```
print("\n" + "=" * 75)
```

```
print("FINAL VULNERABILITY ASSESSMENT")
```

```
print("=" * 75)
```

```
print("\nSystem Name    :", system_info["system_name"])
```

```
print("Total Findings   :", total_vulnerabilities)
```

```
print("High Risk        :", high_count)
```

```
print("Medium Risk       :", medium_count)
```

```
print("Low Risk          :", low_count)
```

```
print("Overall Risk      :", overall_risk)
```

```
if overall_risk == "HIGH":
```

```
print("""
SECURITY STATUS:
HIGH RISK
```

```

Immediate attention should be given to the identified
high-severity vulnerabilities. Services, software versions,
authentication policies, firewall configuration, and
data-protection controls should be reviewed and remediated.
""")
```

```
elif overall_risk == "MEDIUM":
```

```

print("""
SECURITY STATUS:
MEDIUM RISK
```

```

The identified weaknesses should be reviewed and addressed
according to their priority.
""")
```

```
else:
```

```

print("""
SECURITY STATUS:
LOW RISK
```

```

Continue regular vulnerability assessment and security monitoring.
""")
```

```
print("=" * 75)
```

```
print("VULNERABILITY ASSESSMENT COMPLETED SUCCESSFULLY")
```

```
print("=" * 75)
```

```
print("""
```

NOTE:

This practical uses synthetic system information for educational purposes. No real system, website, server, IP address, or network was scanned.

```
""")
```

Expected Output

The program will identify findings such as:

Category	Finding	Severity	Risk
Network Service	Telnet service enabled	HIGH	8
Database Exposure	Database service exposed	HIGH	8
Software Version	Outdated Apache version	HIGH	8
Software Version	Outdated SSH version	MEDIUM	6
Software Version	Outdated MySQL version	HIGH	8
Authentication	Weak password length	MEDIUM	6

Authentication	Password complexity disabled	MEDIUM	6
Network Security	Firewall disabled	HIGH	8
Patch Management	Automatic updates disabled	MEDIUM	6
Data Protection	Encryption disabled	HIGH	8
Access Control	Excessive administrative accounts	MEDIUM	6

The exact total is deterministic with the supplied synthetic data.

Student Tasks

Task 1 – Change Password Policy

Change:

"minimum_password_length": 6

to:

"minimum_password_length": 12

Run the program again and observe the vulnerability report.

Task 2 – Enable Firewall

Change:

"firewall_enabled": False

to:

"firewall_enabled": True

Observe the change in the vulnerability count.

Task 3 – Enable Automatic Updates

Change:

"automatic_updates": False

to:

"automatic_updates": True

Compare the results before and after remediation.

Task 4 – Remove an Insecure Service

Remove the Telnet service from the **services** list and execute the program again.

Observe the change in **HIGH severity vulnerabilities**.

Task 5 – Reduce Administrative Accounts

Change:

"admin_accounts": 7

to:

"admin_accounts": 2

Analyze the effect on the assessment report.

Observation Table

Sr. No.	Security Check	Observation	Severity	Recommended Action
1	Open Port Assessment	Insecure service identified	High	Disable unnecessary service
2	Service Version	Outdated software detected	High/Medium	Upgrade software
3	Password Policy	Weak password configuration	Medium	Strengthen password policy
4	Firewall	Firewall disabled	High	Enable firewall
5	Patch Management	Automatic updates disabled	Medium	Enable/update regularly

6	Encryption	Encryption disabled	High	Enable encryption
7	Privilege Management	Excessive admin accounts	Medium	Apply least privilege

Result

The vulnerability assessment was successfully performed using **Python in Google Colab**. The simulated system configuration was analyzed for insecure services, outdated software, weak authentication settings, firewall configuration, patch-management issues, encryption weaknesses, and excessive administrative privileges. The identified vulnerabilities were classified according to severity, assigned basic risk scores, and presented through an assessment and remediation report.

Conclusion

This practical demonstrates the basic workflow of **vulnerability assessment** in a controlled environment. Students learn how security weaknesses can be systematically identified, categorized, prioritized, and documented. The practical also establishes the relationship between **vulnerability identification, risk assessment, and remediation**, which forms an important foundation for security auditing and penetration-testing concepts.